

測量 Interaction to Next Paint (INP)

來源：<https://codelabs.developers.google.com/measuring-inp?hl=zh-tw>

步驟數：9

目錄

1. [簡介](#)
2. [做好準備](#)
3. [熟悉 Chrome 開發人員工具](#)
4. [安裝 web-vitals](#)
5. [歸因包含哪些內容？](#)
6. [多個事件監聽器](#)
7. [輸入延遲](#)
8. [呈現延遲：更新無法繪製時](#)
9. [結論](#)

步驟 1 · 約 0 分鐘

1. 簡介

本程式碼研究室會以互動方式，說明如何使用 `web-vitals` 程式庫測量 [Interaction to Next Paint \(INP\)](#)。

必要條件

- 熟悉 HTML 和 JavaScript 開發。
- 建議：詳閱 [web.dev 的 INP 指標說明文件](#)。

學習目標

- 如何在網頁中新增 `web-vitals` 程式庫，並使用其歸因資料。
- 使用歸因資料診斷要從何處著手，以及如何開始改善 INP。

軟體需求

- 電腦必須能夠從 GitHub 複製程式碼，並執行 npm 指令。
 - 文字編輯器。
 - 使用新版 Chrome，才能正常進行所有互動評估。
-

步驟 2 · 約 0 分鐘

2. 做好準備

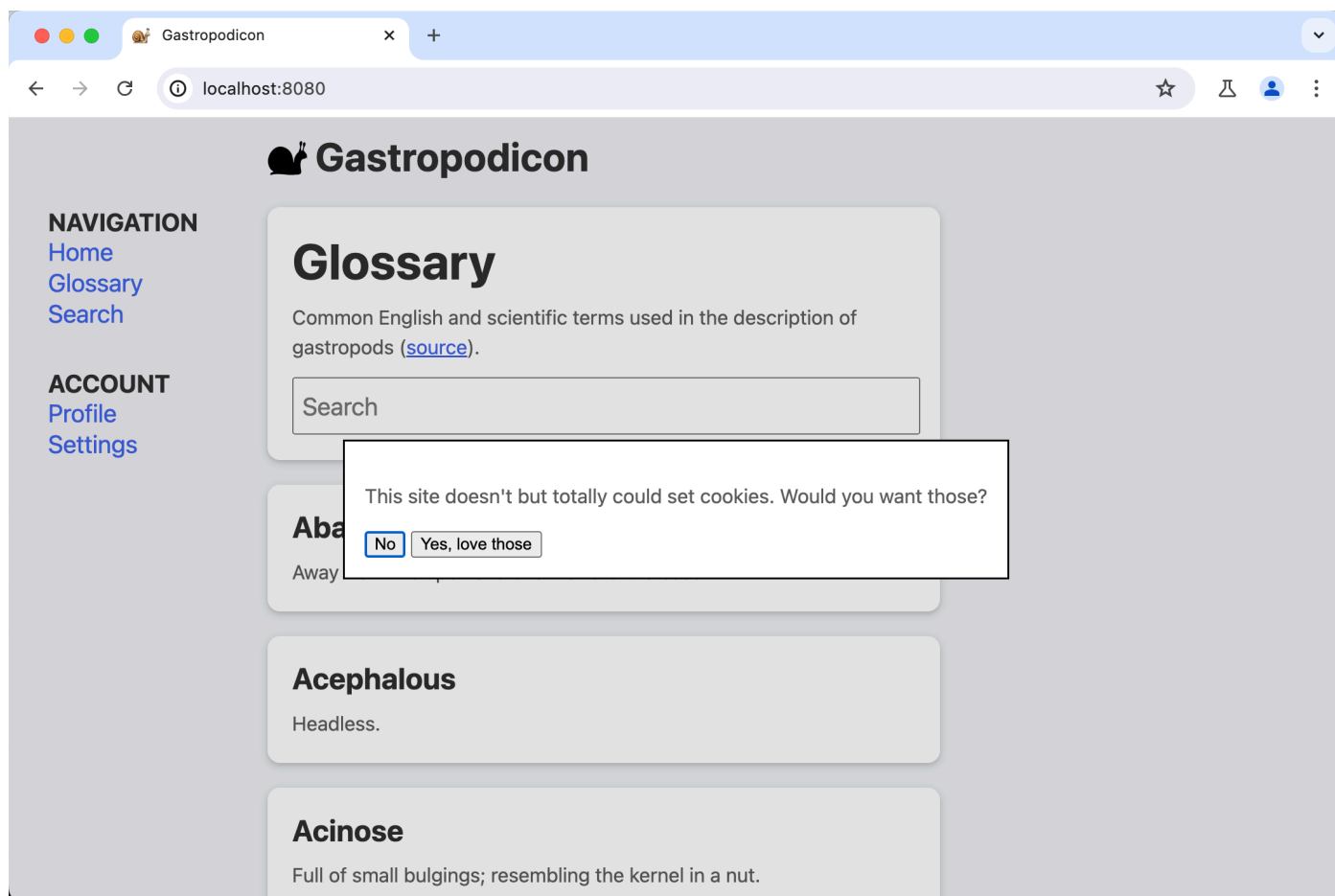
取得並執行程式碼

程式碼位於 [web-vitals-codelabs](#) 存放區。

1. 在終端機中複製存放區：`git clone https://github.com/GoogleChromeLabs/web-vitals-codelabs.git`。
2. 前往複製的目錄：`cd web-vitals-codelabs/measuring-inp`。
3. 安裝依附元件：`npm ci`。
4. 啟動網路伺服器：`npm run start`。
5. 在瀏覽器中前往 <http://localhost:8080/>。

試用頁面

本程式碼研究室會使用 Gastropodicon (熱門的蝸牛解剖學參考網站)，探討 INP 的潛在問題。



嘗試與網頁互動，瞭解哪些互動速度緩慢。

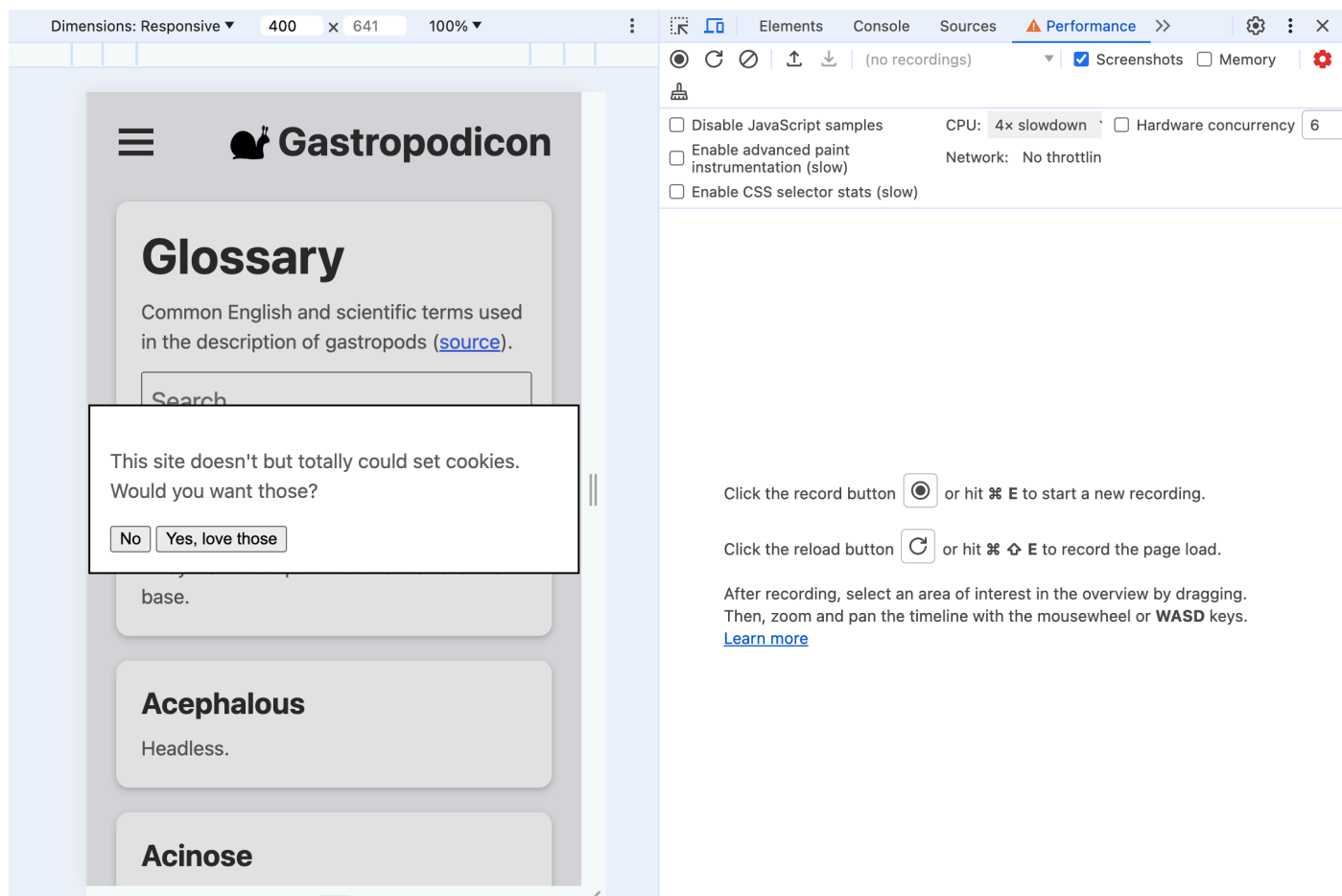
步驟 3 · 約 0 分鐘

3. 熟悉 Chrome 開發人員工具

開啟開發人員工具：依序選取「更多工具」 > 「開發人員工具」選單；在網頁上按一下滑鼠右鍵，然後選取「檢查」；或使用鍵盤快速鍵。

在本程式碼研究室中，我們會使用「效能」面板和「控制台」。您隨時可以透過開發人員工具頂端的分頁標籤切換。

- INP 問題最常發生在行動裝置上，因此請[切換至行動裝置顯示模擬](#)。
- 如果您在桌機或筆電上測試，效能可能會比在實際行動裝置上好得多。如要更貼近實際情況地查看效能，請按一下「效能」面板右上角的齒輪圖示，然後選取「CPU 減速 4 倍」。



如果方便，也可以同時開啟兩個面板。切換至 **效能** 面板，然後按下 **Escape** 鍵 開啟控制台。

步驟 4 · 約 0 分鐘

4. 安裝 web-vitals

`web-vitals` 是一個 JavaScript 程式庫，可評估使用者體驗的 Web Vitals 指標。您可以使用程式庫擷取這些值，然後將這些值信號傳送至數據分析端點，以供後續分析，目的是找出發生緩慢互動的時間和地點。

[新增程式庫至網頁的方法有很多種](#)。在自家網站上安裝程式庫的方式，取決於您管理依附元件的方式、建構程序和其他因素。如要瞭解所有選項，請務必查看程式庫的文件。

本程式碼研究室會從 npm 安裝並直接載入指令碼，避免深入探討特定建構程序。

您可以使用以下兩個版本的 `web-vitals`：

- 如要追蹤網頁載入時的 Core Web Vitals 指標值，請使用「標準」建構版本。
- 「歸因」建構版本會在每個指標中加入額外的偵錯資訊，以診斷指標最終值的原因。

在本程式碼研究室中，我們需要歸因分析建構版本，才能評估 INP。

執行 `npm install -D web-vitals`，將 `web-vitals` 新增至專案的 `devDependencies`

將 `web-vitals` 新增至頁面：

在 `index.html` 底部新增歸因版本的指令碼，並將結果記錄到控制台：

```
<script type="module">
  import {onINP} from './node_modules/web-vitals/dist/web-vitals.attribution.js';

  onINP(console.log);
</script>
```

立即試用

開啟控制台後，請再次與網頁互動。您在頁面上點選任何項目時，系統都不會記錄任何內容！

INP 是在網頁的整個生命週期中測量，因此根據預設，`web-vitals` 會等到使用者離開或關閉網頁時，才回報 INP。這是信號傳送的理想行為，適用於分析等用途，但不太適合用於互動式偵錯。

`web-vitals` 提供 [the `reportAllChanges` 選項](#)，可產生更詳細的報表。啟用後，系統不會回報所有互動，但只要有互動比先前的互動慢，就會回報。

請嘗試將選項新增至指令碼，然後再次與網頁互動：

```
<script type="module">
  import {onINP} from './node_modules/web-vitals/dist/web-vitals.attribution.js';

  onINP(console.log, {reportAllChanges: true});
</script>
```

重新整理頁面後，系統應該就會將互動回報給控制台，並在出現新的最慢互動時更新。舉例來說，請試著在搜尋框中輸入內容，然後刪除輸入的內容。

Dimensions: Responsive ▾400x641100% ▾

Elements

Console

Sources

Network

Performance >>

⚙️

⋮

×

🔍

🔗

top ▾

👁️

Filter

Default levels ▾

No Issues

4 hidden ⚙️

☰

 **Gastropodicon**

Glossary

Common English and scientific terms used in the description of gastropods ([source](#)).

Search

Bicuspid or bicuspidate

Having two cusps.

Cilium (plural cilia)

A lash; used to designate the hairs on the mantle, gills, etc.

Planispiral shell

web-vitals.attribution.js:1

> {name: 'INP', value: 32, rating: 'good', delta: 32, entries: Array(1), ...}

web-vitals.attribution.js:1

> {name: 'INP', value: 560, rating: 'poor', delta: 528, entries: Array(1), ...}

web-vitals.attribution.js:1

▼ {name: 'INP', value: 696, rating: 'poor', delta: 136, entries: Array(1), ...} ⓘ

> attribution: {interactionTarget: '#search-terms', interactionTargetElement: in

delta: 136

> entries: [PerformanceEventTiming]

id: "v4-1715704899419-7631921820701"

name: "INP"

navigationType: "reload"

rating: "poor"

value: 696

> [[Prototype]]: Object

>

5. 歸因包含哪些內容？

首先，我們來看看大多數使用者與網頁的第一次互動，也就是 Cookie 同意聲明對話方塊。

許多網頁都有需要同步觸發 Cookie 的指令碼，因此使用者接受 Cookie 後，點擊就會變成緩慢的互動。這就是這裡的情況。

如果已關閉 Cookie 同意聲明，您可能需要清除示範的本機儲存空間，才能再次看到聲明。在瀏覽器的網址列中，按一下網址旁的資訊圖示，選取「Cookie 和網站資料」，然後選取「管理裝置端網站資料」，接著按一下「localhost」旁的垃圾桶圖示即可清除。重新整理頁面，Cookie 同意聲明對話方塊應該會再次顯示。

按一下「Yes」接受 (示範) Cookie，並查看現在記錄到開發人員工具控制台的 INP 資料。

```
web-vitals.attribution.js:1
▼ {name: 'INP', value: 344, rating: 'needs-improvement', delta: 344, entries: Array(1), ...} ⓘ
  ► attribution: {interactionTarget: '#confirm', interactionTargetElement: button#confirm, inter
    delta: 344
  ► entries: [PerformanceEventTiming]
    id: "v4-1715732159298-8028729544485"
    name: "INP"
    navigationType: "reload"
    rating: "needs-improvement"
    value: 344
  ► [[Prototype]]: Object
```

標準和歸因網頁指標建構版本都會提供這項頂層資訊：

```
{
  name: 'INP',
  value: 344,
  rating: 'needs-improvement',
  entries: [...],
  id: 'v4-1715732159298-8028729544485',
  navigationType: 'reload',
  attribution: {...},
}
```

從使用者點選到下一次繪製的時間長度為 344 毫秒，屬於「需要改善」的 INP。entries 陣列包含與這項互動相關的所有 PerformanceEntry 值，在本例中只有一個點擊事件。

不過，如要瞭解這段時間的狀況，我們最感興趣的是 attribution 資源。為建構歸因資料，web-vitals 會找出與點擊事件重疊的長時間動畫影格 (LoAF)。LoAF 隨後會提供詳細資料，說明該影格期間的時間分配情形，包括執行的指令碼，以及在 requestAnimationFrame 回呼、樣式和版面配置中花費的時間。

展開 attribution 屬性即可查看更多資訊。資料更豐富。


```
attribution: {
  interactionTargetElement: Element,
  interactionTarget: '#confirm',
  interactionType: 'pointer',

  inputDelay: 27,
  processingDuration: 295.6,
  presentationDelay: 21.4,

  processedEventEntries: [...],
  longAnimationFrameEntries: [...],
}
```

首先是互動內容的相關資訊：

- `interactionTargetElement`：與之互動的元素即時參照 (如果該元素尚未從 DOM 中移除)。
- `interactionTarget`：用於在網頁中尋找元素的選取器。

接著，我們將時間劃分為幾個階段：

- `inputDelay`：使用者開始互動 (例如點選滑鼠) 到該互動的事件監聽器開始執行的時間差。在本例中，即使 CPU 節流開啟，輸入延遲也只有約 27 毫秒。
- `processingDuration`：事件監聽器執行完畢所需的時間。通常網頁會為單一事件設定多個監聽器 (例如 `pointerdown`、`pointerup` 和 `click`)。如果這些監聽器都在同一個動畫影格中執行，就會合併到這個時間。在本例中，處理時間為 295.6 毫秒，占 INP 時間的大宗。
- `presentationDelay`：從事件監聽器完成作業，到瀏覽器完成繪製下一個影格的時間。在本例中為 21.4 毫秒。

這些 INP 階段是診斷需要最佳化項目的重要信號。如要進一步瞭解這個主題，請參閱[最佳化 INP 指南](#)。

再深入一點，`processedEventEntries` 包含 五個事件，而非頂層 INP `entries` 陣列中的單一事件。其中有何區別？

```

processedEventEntries: [
  {
    name: 'mouseover',
    entryType: 'event',
    startTime: 1801.6,
    duration: 344,
    processingStart: 1825.3,
    processingEnd: 1825.3,
    cancelable: true
  },
  {
    name: 'mousedown',
    entryType: 'event',
    startTime: 1801.6,
    duration: 344,
    processingStart: 1825.3,
    processingEnd: 1825.3,
    cancelable: true
  },
  {name: 'mousedown', ...},
  {name: 'mouseup', ...},
  {name: 'click', ...},
],

```

頂層項目是 INP 事件，在本例中為點擊。歸因 `processedEventEntries` 是在同一影格處理的所有事件。請注意，其中包含 `mouseover` 和 `mousedown` 等其他事件，而不只是點擊事件。如果這些其他事件也很緩慢，瞭解這些事件就至關重要，因為這些事件都會導致回應速度緩慢。

最後是 `longAnimationFrameEntries` 陣列。這可能只是單一項目，但有時互動會跨越多個影格。這是最簡單的情況，只有一個長動畫影格。

`longAnimationFrameEntries`

展開 LoAF 項目：

```

longAnimationFrameEntries: [{
  name: 'long-animation-frame',
  startTime: 1823,
  duration: 319,

  renderStart: 2139.5,
  styleAndLayoutStart: 2139.7,
  firstUIEventTimestamp: 1801.6,
  blockingDuration: 268,

  scripts: [{...}]
}],

```

這裡有許多實用值，例如樣式設定所花費的時間量。[「Long Animation Frames API」一文會更深入探討這些屬性](#)。目前我們主要感興趣的是 `scripts` 屬性，其中包含的項目會提供導致影格長時間執行的指令碼詳細資料：

```
scripts: [{
  name: 'script',
  invoker: 'BUTTON#confirm.onclick',
  invokerType: 'event-listener',

  startTime: 1828.6,
  executionStart: 1828.6,
  duration: 294,

  sourceURL: 'http://localhost:8080/third-party/cmp.js',
  sourceFunctionName: '',
  sourceCharPosition: 1144
}]
```

在本例中，我們可以得知時間主要花在 `BUTTON#confirm.onclick` 上叫用的單一 `event-listener`。我們甚至可以看到指令碼來源網址，以及函式定義位置的字元位置！

外帶

從這項歸因資料，可以判斷這個案件的哪些資訊？

- 互動是由點選 `button#confirm` 元素 (來自 `attribution.interactionTarget` 和指令碼歸因項目的 `invoker` 屬性) 觸發。
- 主要時間用於執行事件監聽器 (相較於總指標 `attribution.processingDuration`)。 `value`
- 緩慢的事件監聽器程式碼會從 `third-party/cmp.js` (來自 `scripts.sourceURL`) 中定義的點擊事件監聽器開始。

這就足以瞭解需要進行最佳化的部分！

步驟 6 · 約 0 分鐘

6. 多個事件監聽器

重新整理頁面，清除開發人員工具控制台，這樣 Cookie 同意聲明互動就不會再是最長的互動。

在搜尋框中輸入查詢。歸因資料會顯示什麼？你認為發生了什麼事？

歸因分析資料

首先，請先大致掃描測試試用版的一個範例：

```
{
  name: 'INP',
  value: 1072,
  rating: 'poor',
  attribution: {
    interactionTargetElement: Element,
    interactionTarget: '#search-terms',
    interactionType: 'keyboard',

    inputDelay: 3.3,
    processingDuration: 1060.6,
    presentationDelay: 8.1,

    processedEventEntries: [...],
    longAnimationFrameEntries: [...],
  }
}
```

這是與 `input#search-terms` 元素進行鍵盤互動時，INP 值不佳 (已啟用 CPU 節流) 的情況。大部分時間 (1061 毫秒，INP 總時間為 1072 毫秒) 都用於處理程序。

不過，`scripts` 的條目更有趣。

版面配置輾轉現象

`scripts` 陣列的第一個項目提供了一些實用情境：

```
scripts: [{
  name: 'script',
  invoker: 'BUTTON#confirm.onclick',
  invokerType: 'event-listener',

  startTime: 4875.6,
  executionStart: 4875.6,
  duration: 497,
  forcedStyleAndLayoutDuration: 388,

  sourceURL: 'http://localhost:8080/js/index.js',
  sourceFunctionName: 'handleSearch',
  sourceCharPosition: 940
},
...]
```

大部分的處理時間都發生在這個指令碼執行期間，也就是 `input` 監聽器 (呼叫者為 `INPUT#search-terms.oninput`)。函式名稱 (`handleSearch`) 和 `index.js` 來源檔案中的字元位置都會顯示。

不過，現在有新的屬性： `forcedStyleAndLayoutDuration`。這是指瀏覽器強制重新排版網頁時，在這個指令碼叫用中花費的時間。換句話說，執行這個事件監聽器所花費的時間有 78% (497 毫秒中的 388 毫秒) 實際上都浪費在版面配置抖動上。

應優先修正這個問題。

重複收聽者

就個別而言，接下來的兩個指令碼項目沒有特別之處：

```
scripts: [...,  
{  
  name: 'script',  
  invoker: '#document.onkeyup',  
  invokerType: 'event-listener',  
  
  startTime: 5375.3,  
  executionStart: 5375.3,  
  duration: 124,  
  
  sourceURL: 'http://localhost:8080/js/index.js',  
  sourceFunctionName: '',  
  sourceCharPosition: 1526,  
},  
{  
  name: 'script',  
  invoker: '#document.onkeyup',  
  invokerType: 'event-listener',  
  
  startTime: 5673.9,  
  executionStart: 5673.9,  
  duration: 95,  
  
  sourceURL: 'http://localhost:8080/js/index.js',  
  sourceFunctionName: '',  
  sourceCharPosition: 1526  
}]
```

這兩項都是 `keyup` 監聽器，會依序執行。監聽器是匿名函式 (因此 `sourceFunctionName` 屬性中不會回報任何內容)，但我們仍有來源檔案和字元位置，因此可以找出程式碼所在位置。

奇怪的是，兩者都來自同一個來源檔案和字元位置。

瀏覽器最後在單一動畫影格中處理多個按鍵事件，導致這個事件監聽器在任何內容繪製完成前執行了兩次！

這種影響也可能加劇，事件監聽器完成時間越長，可能傳入的額外輸入事件就越多，導致互動時間越長。

由於這是搜尋/自動完成互動，因此延遲輸入是個好策略，這樣每個影格最多只會處理一個按鍵。

7. 輸入延遲

輸入延遲的常見原因 (使用者互動到事件監聽器開始處理互動之間的時間) 是因為主要執行緒忙碌。可能原因如下：

- 網頁正在載入，主要執行緒忙於執行初始工作，包括設定 DOM、配置網頁版面和設定樣式，以及評估和執行指令碼。
- 網頁通常很忙碌，例如執行運算、指令碼型動畫或廣告。
- 先前的互動處理時間過長，導致後續互動延遲，如上一個範例所示。

示範網頁有一個隱藏功能，只要點選頁面頂端的蝸牛標誌，就會開始動畫，並執行一些耗用大量主執行緒資源的 JavaScript 工作。

- 按一下蝸牛標誌即可開始播放動畫。
- 當蝸牛位於彈跳的底部時，系統會觸發 JavaScript 工作。請盡量在跳出前與網頁互動，看看能觸發多高的 INP。

舉例來說，即使您未觸發任何其他事件監聽器 (例如在蝸牛彈跳時點選並聚焦搜尋框)，主執行緒工作仍會導致網頁在一段時間內沒有回應。

在許多網頁上，繁重的主執行緒工作不會這麼順暢，但這項示範可清楚說明如何在 INP 歸因資料中識別這類工作。

以下是僅在蝸牛跳出期間聚焦搜尋框的歸因範例：

```
{
  name: 'INP',
  value: 728,
  rating: 'poor',

  attribution: {
    interactionTargetElement: Element,
    interactionTarget: '#search-terms',
    interactionType: 'pointer',

    inputDelay: 702.3,
    processingDuration: 4.9,
    presentationDelay: 20.8,

    longAnimationFrameEntries: [{
      name: 'long-animation-frame',
      startTime: 2064.8,
      duration: 790,

      renderStart: 2065,
      styleAndLayoutStart: 2854.2,
      firstUIEventTimestamp: 0,
      blockingDuration: 740,

      scripts: [{...}]
    }]
  }
}
```

如預期，事件監聽器執行速度很快，處理時間為 4.9 毫秒，而絕大多數的互動不良情形都發生在輸入延遲，在總共 728 毫秒中佔了 702.3 毫秒。

這種情況的偵錯難度比較大。雖然我們知道使用者與哪些內容互動，以及互動方式，但我們也知道互動的這部分很快就完成，沒有問題。而是頁面上的其他項目延遲了互動的處理程序，但我們該從何著手呢？

LoAF 指令碼項目可解決這個問題：

```
scripts: [{
  name: 'script',
  invoker: 'SPAN.onanimationiteration',
  invokerType: 'event-listener',

  startTime: 2065,
  executionStart: 2065,
  duration: 788,

  sourceURL: 'http://localhost:8080/js/index.js',
  sourceFunctionName: 'cryptodaphneCoinHandler',
  sourceCharPosition: 1831
}]
```

即使這個函式與互動無關，但確實會減緩動畫影格速度，因此會納入與互動事件合併的 LoAF 資料。

從這裡，我們可以瞭解觸發互動處理延遲的函式 (透過 `animationiteration` 事件監聽器)、負責的確切函式，以及該函式在來源檔案中的位置。

8. 呈現延遲：更新無法繪製時

呈現延遲是指事件監聽器執行完畢後，到瀏覽器能夠在畫面上繪製新影格，向使用者顯示可見回饋之間的時間。

重新整理網頁，再次重設 INP 值，然後開啟漢堡選單。開啟時會明顯頓挫。

這看起來會是什麼樣子？

```
{
  name: 'INP',
  value: 376,
  rating: 'needs-improvement',
  delta: 352,

  attribution: {
    interactionTarget: '#sidenav-button>svg',
    interactionType: 'pointer',

    inputDelay: 12.8,
    processingDuration: 14.7,
    presentationDelay: 348.5,

    longAnimationFrameEntries: [{
      name: 'long-animation-frame',
      startTime: 651,
      duration: 365,

      renderStart: 673.2,
      styleAndLayoutStart: 1004.3,
      firstUIEventTimestamp: 138.6,
      blockingDuration: 315,

      scripts: [{...}]
    }]
  }
}
```

這次造成互動緩慢的主要原因是 *呈現延遲*。也就是說，阻斷主要執行緒的任何項目，都會在事件監聽器完成後發生。

```
scripts: [{
  entryType: 'script',
  invoker: 'FrameRequestCallback',
  invokerType: 'user-callback',

  startTime: 673.8,
  executionStart: 673.8,
  duration: 330,

  sourceURL: 'http://localhost:8080/js/side-nav.js',
  sourceFunctionName: '',
  sourceCharPosition: 1193,
}]
```

查看 `scripts` 陣列中的單一項目，我們可以看到時間花費在 `FrameRequestCallback` 中的 `user-callback`。這次的顯示延遲是由 `requestAnimationFrame` 回呼所造成。

由於 `requestAnimationFrame` 通常用於在每個影格上觸發 JavaScript 以進行動畫處理，因此很容易將其視為觸發下一個影格的回呼，但實際上，它是要求在瀏覽器繪製目前影格之前的回呼。也就是說，如果互動觸發的 `requestAnimationFrame` 回呼速度緩慢，該互動就會變慢，且更有可能導致 INP 偏低。

9. 結論

匯總欄位資料

值得注意的是，如果只查看單一載入網頁的單一 INP 歸因項目，這一切都會變得簡單許多。如何彙整這項資料，根據欄位資料偵錯 INP？有用的詳細資料越多，反而越難達成目標。

舉例來說，瞭解哪個網頁元素是互動緩慢的常見原因，就非常實用。不過，如果網頁已編譯的 CSS 類別名稱會因建構而異，則相同元素的 `web-vitals` 選取器在不同建構版本中可能會有所不同。

您必須考量特定應用程式，判斷最實用的資料以及資料的匯總方式。舉例來說，在回傳信標歸因資料前，您可以根據目標所在的元件，或目標滿足的 [ARIA 角色](#)，將 `web-vitals` 選取器替換為自己的 ID。

同樣地，`scripts` 項目可能在 `sourceURL` 路徑中含有檔案式雜湊，導致難以合併，但您可以在將資料傳回信標之前，根據已知的建構程序移除雜湊。

很遺憾，這麼複雜的資料沒有簡單的處理方式，但即使只使用部分資料，也比完全沒有歸因資料更有價值，有助於進行偵錯程序。

在各處註明出處！

以 LoAF 為基礎的 INP 歸因是強大的偵錯輔助工具。這項 API 可提供詳細資料，說明 INP 期間發生的具體情況。在許多情況下，這項工具會指出指令碼中應開始最佳化工作的精確位置。

現在您可以在任何網站上使用 INP 歸因資料了！

即使您沒有編輯頁面的權限，也可以在開發人員工具控制台中執行下列程式碼片段，重現本程式碼研究室的程序，看看能找到什麼：

```
const script = document.createElement('script');
script.src = 'https://unpkg.com/web-vitals@4/dist/web-vitals.attribution.iife.js';
script.onload = function () {
  webVitals.onINP(console.log, {reportAllChanges: true});
};
document.head.appendChild(script);
```

瞭解詳情

- [web.dev：Interaction to Next Paint](#)
 - [web.dev：找出實際工作環境中速度緩慢的互動](#)
 - [web.dev：最佳化 Interaction to Next Paint](#)
 - [瞭解 INP 程式碼研究室](#)
-